AUGMENTED SOFTWARE ENGINEERING

STATE-OF-THE-UNION @ ONEVIEW

11-MAR-2027

ROLAND@TRITSCH.EMAIL

WELCOME/OBJECTIVE

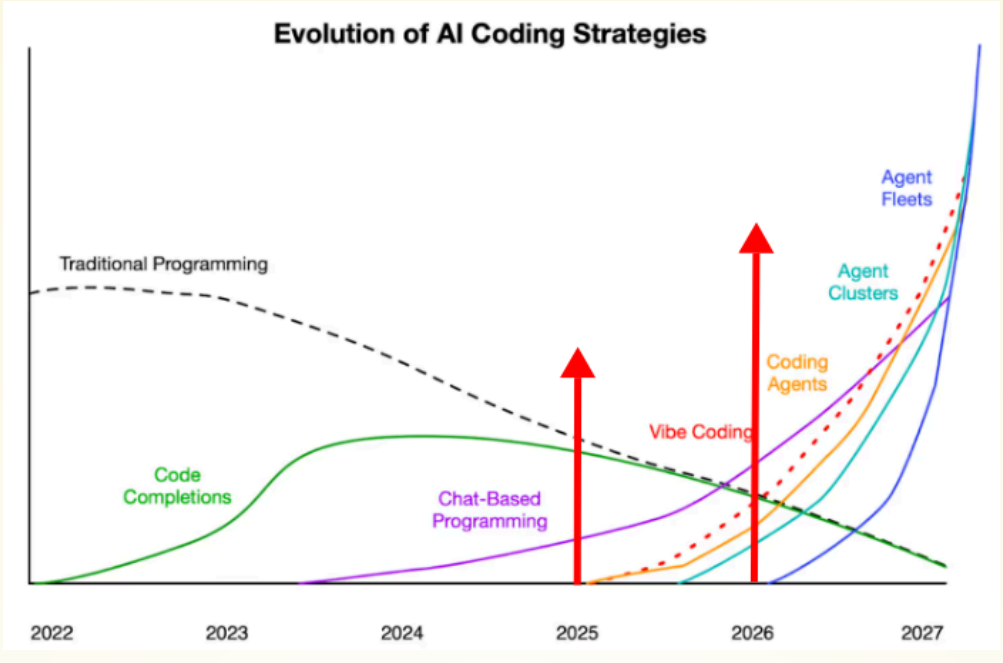
- Augmented Software-Engineering
- Skate to where the puck is going to be
- Today, this year and beyond
- The opportunities and the challenges

Source:
https://en.wikipedia.org/wiki/Wayne_Gretzk

Speaker notes

- Why are we here in this room (besides that Declan and myself bumped into each other at the SonaLake Christmas Party and besides that I am running the ASE meetup)?
- Who knows who Wayne Gretzky is?
    - (Most) Legendary Hockey Player of all times
    - When he was 40 he was still playing and was still better than most of the the other 20-something players. A reporter asked him: How do you do it? I never skate to where the puck is. I always skate to where the puck is going to be!
- Let's (try to) skate to …

OBJECTIVE/MOTIVATION/WHY

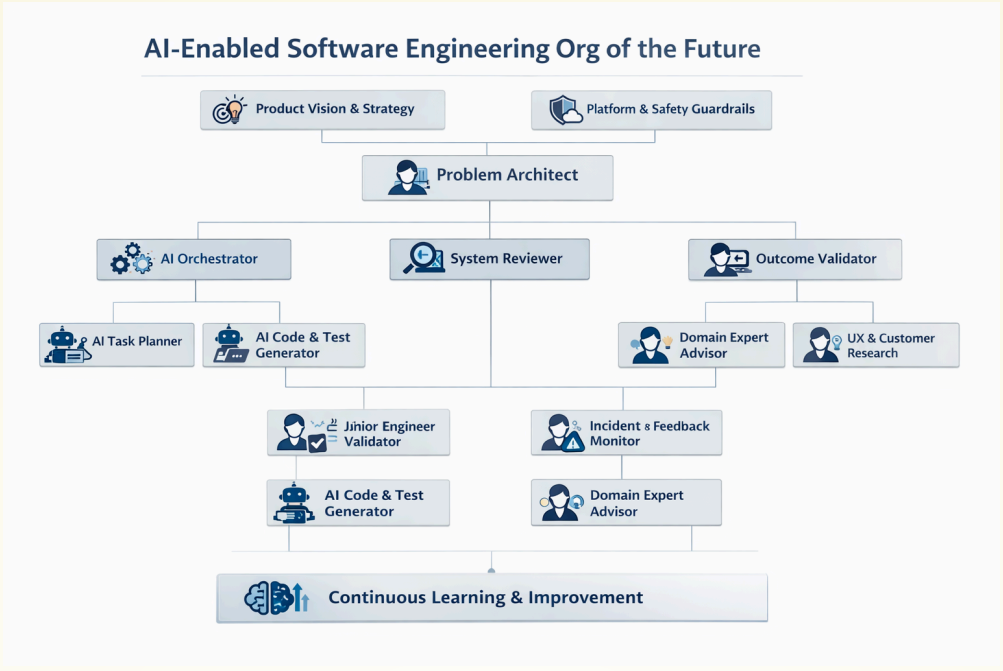


Source: https://sourcegraph.com/blog/revenge-of-the-junior-developer

Speaker notes

- He is an opinionated (sometimes controversial) software-engineer who (last year) wrote a blog-post about the revenge of the junior software engineer
- This was an answer to the premise that junior software engineeer will be replaced by AI. His answer was: Maybe not. Maybe they will replace senior-engineers that are not ai-enabled. Let's wait and see …
- In this article he was also showing this picture. First we had super-IDEs (with super tab-completion). Now we have/do vibe-coding. Next are agent clusters/fleets. Let's talk about this later …

OBJECTIVE/MOTIVATION/WHY

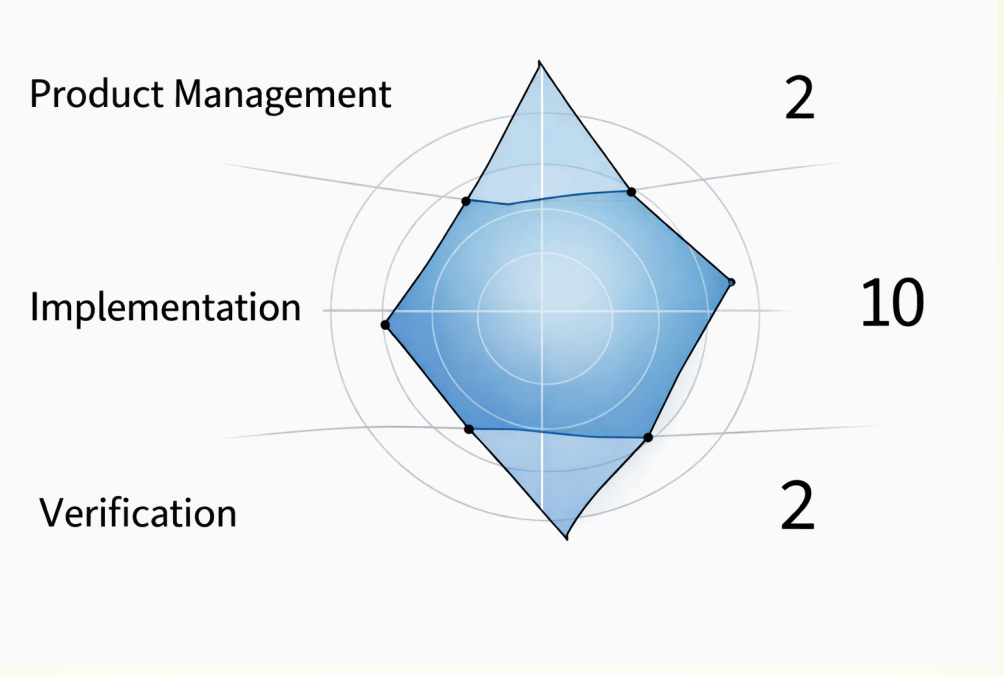


Source: https://tedn.life/2026/02/07/the-augmented-software-organization-moving-on/

Speaker notes

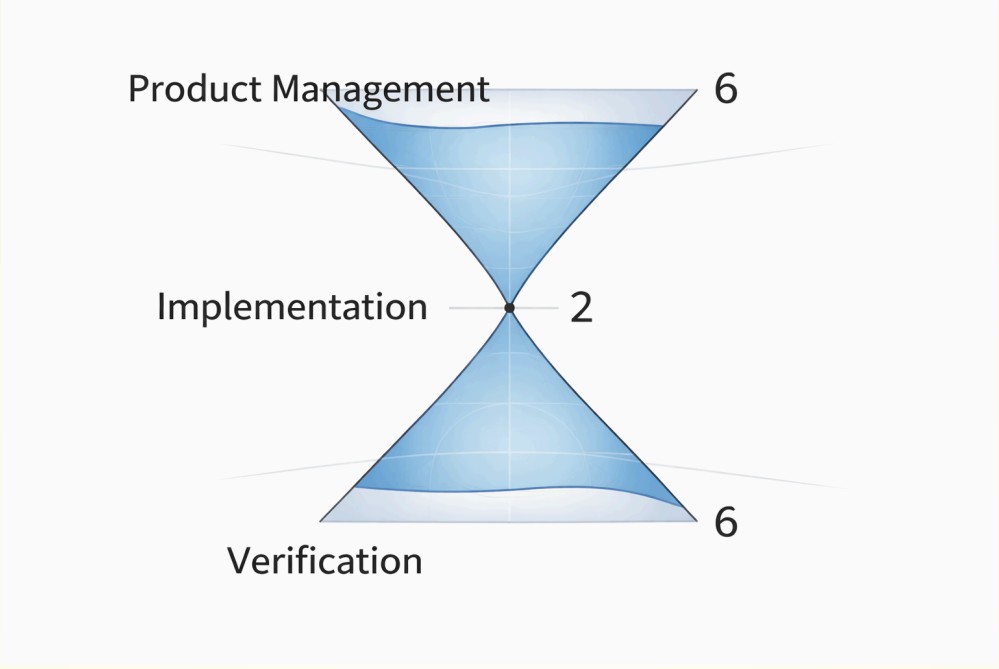
- What does that mean for you? Change is coming! Evolution vs. Revolution?
- Not only for you as individuals, but also for organisations in general
- New processes need to be designed and new roles and responsibilities need to be defined
- Food for thought …

OBJECTIVE/MOTIVATION/WHY



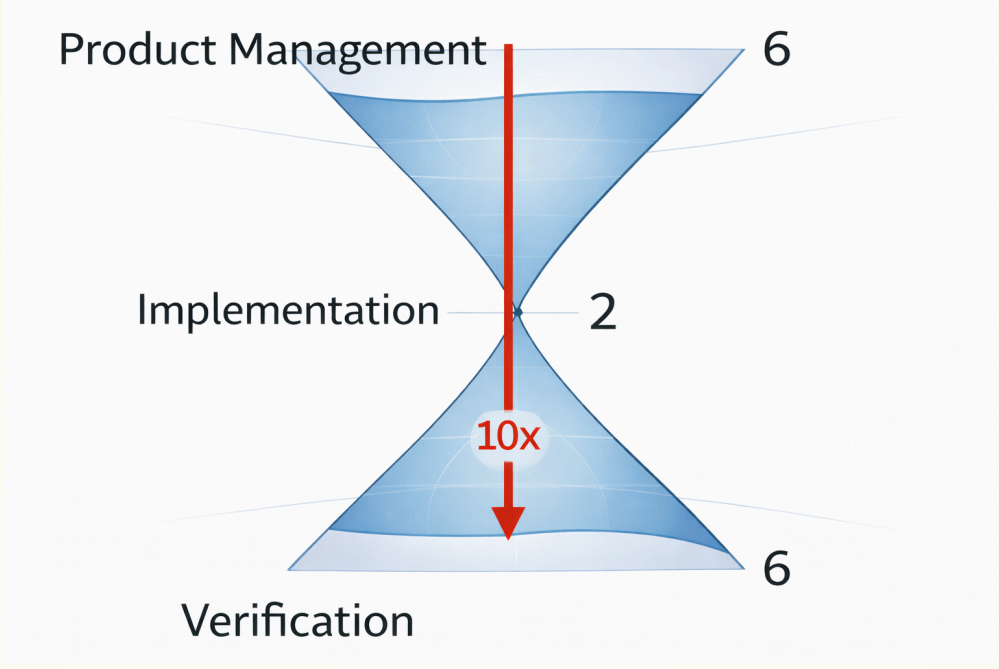
Source: https://tedn.life/2026/02/07/the-augmented-software-organization-moving-on/

OBJECTIVE/MOTIVATION/WHY



Source: https://tedn.life/2026/02/07/the-augmented-software-organization-moving-on/

OBJECTIVE/MOTIVATION/WHY



Source: https://tedn.life/2026/02/07/the-augmented-software-organization-moving-on/

WHO

Roland Tritsch

Extreme Digital Nomad

Biography

Builder and runner of functional teams and functional systems. Explorer of extreme digital nomading and remote-first cultures.

My name is Roland Tritsch. I am a Software Engineer. I am a Manager. I am a Husband and a Father. I like to make engineers grow and users smile. I like computer science. I believe that computers and software can help to make this planet a better place. When I am not building, running or fixing something (preferably using a functional programming language like Haskell or Scala), I am on a hike (somewhere).

You can find/follow me here.

Source: https://tedn.life

Speaker notes

- First things first …
- Who is Roland?
- Career CTO/VP (IONA, Gilt, Nitro, Community, …), Ailtir CTO, …
- Writing code, building systems, leading people, … for 35 years (and counting)
- Passionate about Distributed Systems, Databases, Functional Programming (Scala), (Remote-First) Software-Engineering, …
- Augmented Software Engineering Meetup (on Mar. 26th, 18:00, UCD, L1.03)
- Disclaimer: Opinionated, biased, …, time-constrained (picking random points)

TODAY

- Multispeed adoption
  - individuals vs. organisations
  - tasks (5x) vs. results/impact (0.5x)
  - by size (small(er) eq. fast(er))
  - by industry (regulated)
  - by lifecycle (prototype vs. maintenance)
- It is easy to feel slow :)

Speaker notes

- The single most important thing to understand about AI adoption right now is that it is not one speed — it is many speeds happening simultaneously
- Individuals can adopt a new tool overnight. Organisations need process, governance, risk reviews, procurement, and a strategy. That gap is real and it creates friction and frustration on both sides.
- "Tasks 5x, results 0.5x" — this is the central tension we will unpack in the next few slides. Individual tasks feel dramatically faster. Boilerplate, first drafts, test generation — genuinely faster. But does that translate to shipping more? To better outcomes? The data is… surprising. Hold that thought.
- Size matters: smaller organisations move faster because they have fewer constraints — less legacy code, shorter approval chains, less to lose. The irony is that the organisations with the most to gain from efficiency improvements (large ones) are also the slowest to adopt.
- Regulated industries — finance, healthcare, legal — face a double burden: they need AI to stay competitive AND they face the highest compliance overhead in deploying it. Governance catches up eventually, but there is a lag.
- Lifecycle is underappreciated: AI is transformative for green-field work and prototypes. It is much less effective — and sometimes counterproductive — on large, mature, complex codebases. Where most professional engineers actually spend their time.
- "It is easy to feel slow" — acknowledge the room. There is a lot of noise out there. Announcements every week. Someone on LinkedIn claiming 10x productivity. The honest answer is: you are probably not slow. The field is just moving in multiple directions at once.

TODAY - CHALLENGES

- Low signal to noise ratio (noise is high(er))
- No established adapation/maturity frameworks (for organisations)
- Confusing, Frustrating, Intimitating, Threatening, …

Speaker notes

- **Signal-to-noise:** A new model drops every two weeks. A new tool every week. Every vendor claims 10x productivity. As a Principal or Director, your job is to place bets — and the cost of a wrong bet is not just money, it is three months of your team's workflow built around the wrong thing. There is no Consumer Reports for AI developer tools yet. You are evaluating in motion, while also shipping product, while also keeping the lights on.
- **No established maturity framework:** We have Agile. We have DevOps. We have SRE. These took years to emerge but gave organisations a shared vocabulary and a progression model. For AI-augmented engineering, nothing equivalent is widely adopted yet. DORA — the most credible source in DevOps metrics — only just published an AI Capabilities Model in 2025. That it is brand new tells you everything about where we are. Every org is still inventing it from scratch. What does "we use AI well" even mean? How do you measure it? How do you govern it — security, IP, hallucinated dependencies, data leakage? You cannot hire for it, you cannot benchmark against peers, and you cannot put it in a job description. That is deeply uncomfortable at an organisational scale.
- **The human dimension is the hardest part:** Your senior engineers — the ones who carry the institutional knowledge, who review every PR, who define the architecture — they feel this most acutely. Their expertise is suddenly being questioned. Their juniors are generating code at volume they cannot. The threat is not always explicit, but it is present in every standup and every 1:1. And on the other side: junior engineers are overconfident, shipping AI-generated code they do not fully understand, which lands on the senior engineers to review. The psychological safety of your team is now a first-order engineering problem. That is new.

TODAY - OPPORTUNITIES

- Continue to level-up (as individuals)
- Develop your own adoption framework/style (as organisations)
  - Spec-Driven Design vs. Pair-Programming
  - Standardization, Agents, Agents-Automation
- Invest in `*-as-Code` (to get ready for what comes next)
  - Infrastructure, Architecture, Test (coverage, mutation, …)

Speaker notes

- **Level-up individuals — and do it deliberately:** The Brynjolfsson data showed +34% for novice workers. You have 100 engineers. Even a modest AI fluency uplift across the whole team compounds dramatically. But "everyone gets a Copilot license" is not a strategy — it is a procurement decision. The opportunity is to invest in deliberate practice: pairing sessions, internal demos of what good AI-assisted workflows actually look like, psychological safety to experiment and fail. The engineers who figure out how to combine deep domain expertise with AI leverage will be the most valuable people in the industry. Your job is to create the conditions for that to happen.
- **Define your own adoption framework — now, while the window is open:** Since no industry standard exists yet, the first orgs to crystallise their own approach have a durable advantage. You get to set the benchmark rather than chase it. Concretely: pick a style and go deep on it. Spec-Driven Development — write rigorous specs, use AI to implement — rewards engineers who can think clearly about requirements. AI Pair-Programming — tight human-AI loops in the editor — rewards engineers who can evaluate and steer quickly. These are not the same skill. Know which one fits your codebase, your team, and your culture, and build your tooling and process around it intentionally.
- **Build the groundwork for the next wave now:** Agents and agent automation are not mainstream yet — but they are coming, and the orgs that will benefit most are the ones already doing the things that make agent-driven development possible: clear specs, high test coverage, well-documented architecture, small composable services. The opportunity is that this groundwork is worth doing anyway. You are not over-investing in a bet — you are building good engineering hygiene that will pay off 2x when the next wave arrives.

TODAY - (MY) SUGGESTIONS

Invest in your …

- business-domain know-how
- ability to read/review artifacts (a lot and fast)
- ability to express requirements (what needs to be build)
- ability to express how good/right looks like (definition of done)
- tools and tooling (voice, playwright, mermaid, …)

Speaker notes

- **Business-domain know-how:** AI can write code. It cannot deeply understand your specific domain — your data model, your regulatory constraints, your customers' edge cases, the decade of architectural decisions baked into your system. That context lives in your engineers' heads. The engineer who brings deep domain knowledge to an AI-assisted workflow gets dramatically better output and catches hallucinations before they ship. Domain expertise is becoming the differentiator — not syntax fluency, not knowing which framework. Invest time learning the business, not just the stack.
- **Read and review fast:** When AI generates code at 10x speed, the bottleneck shifts to review. If your team's output doubles but your review capacity stays flat, you have a new crisis. Train the skill of reading code quickly and critically — skimming a 500-line PR in 10 minutes and spotting the subtle logic error on line 340. This applies to more than code: specs, architecture docs, test plans. The ability to process a lot of AI-generated artifact quickly and accurately is one of the highest-leverage skills to develop now.
- **Express requirements precisely:** "Vibe prompting" produces vibe output. The engineers who write precise, unambiguous specifications — who can say exactly what they want and what "done" looks like before a single line is written — get qualitatively better results from AI. This is requirements engineering and product thinking. Invest in structured writing, in the discipline of disambiguation, in the habit of writing acceptance criteria before the prompt. It is the same skill that makes a great tech lead. AI just makes its absence much more expensive.
- **Define "good" explicitly:** Can you articulate the quality bar? Can you write the tests that would catch a wrong answer? Can you describe the edge cases, the failure modes, the non-functional requirements? The human's job is increasingly to define "right," not to implement it. Engineers who have never had to write rigorous acceptance criteria or mutation-test their suites will feel this gap acutely.
- **Tools and tooling — including voice:** Know your tools deeply. Not just which AI assistant, but how to configure it for your codebase, when to use which mode, how to chain tools together. And specifically: learn voice input. The engineers who can dictate a clear spec or a precise prompt while walking around are removing a real friction point. Voice is underrated and underused in this industry. Thirty minutes of practice a day compounds fast.

THIS YEAR - CHALLENGES

- The new burn-out. Too much …
  - reading, context-switching, "change"
- Shifting the time/money budget
  - Learning/Tooling/Doing - 10/20/70 - 20/30/50
- Quality/trust erosion (review fatigue, hidden technical debt)
- Hiring signal collapse (the AI-native junior cohort arrives)

Speaker notes

- **The new burn-out:** This is not the old burn-out of too much work. This is cognitive overload from too much change and too much to process. Every week there is a new tool to evaluate, a new model to understand, a new AI-generated PR to review. Engineers who were already stretched thin now have a second job: keeping up. The reading load alone — AI output, AI-generated docs, AI-written tests — is relentless. Context-switching between your own mental model and an AI's output is more taxing than it looks. Watch your team for this. The engineers going quietest in standups may be the ones drowning.
- **Shifting the budget:** The 10/20/70 split — 10% learning, 20% tooling, 70% doing — is breaking. The orgs that thrive will rebalance toward something like 20/30/50. That means less "doing" in the traditional sense and more investment in capability building. But the business still expects the same output. That gap between what leadership expects and what the team needs to invest in is a real management challenge for the next 12-18 months. You have to make the case for that rebalancing with data, not just instinct.
- **Quality and trust erosion:** AI-generated code is plausible. It looks right. It often passes CI. The subtle logic error, the missing edge case, the hallucinated API call that happens to compile — these slip through. As AI output floods your PRs, reviewer fatigue sets in. "LGTM culture" on AI-generated code is a real and growing risk. Within 12-18 months, the teams that moved fast without investing in review rigour will start hitting the accumulated debt. Security audits, production incidents, and compliance reviews will surface it. The challenge is that the debt is invisible until it is not.
- **Hiring signal collapse:** The first cohort of engineers who learned to code primarily with AI assistance is entering the workforce now. The traditional hiring signals — LeetCode, GitHub contributions, side projects, take-home assignments — are all trivially AI-generatable. How do you assess whether someone actually understands what they built? How do you differentiate a strong engineer from a skilled AI operator? The industry does not have a good answer yet. In the next 12-18 months this becomes an acute problem as hiring pipelines fill with candidates whose output looks polished but whose underlying model is shallow.

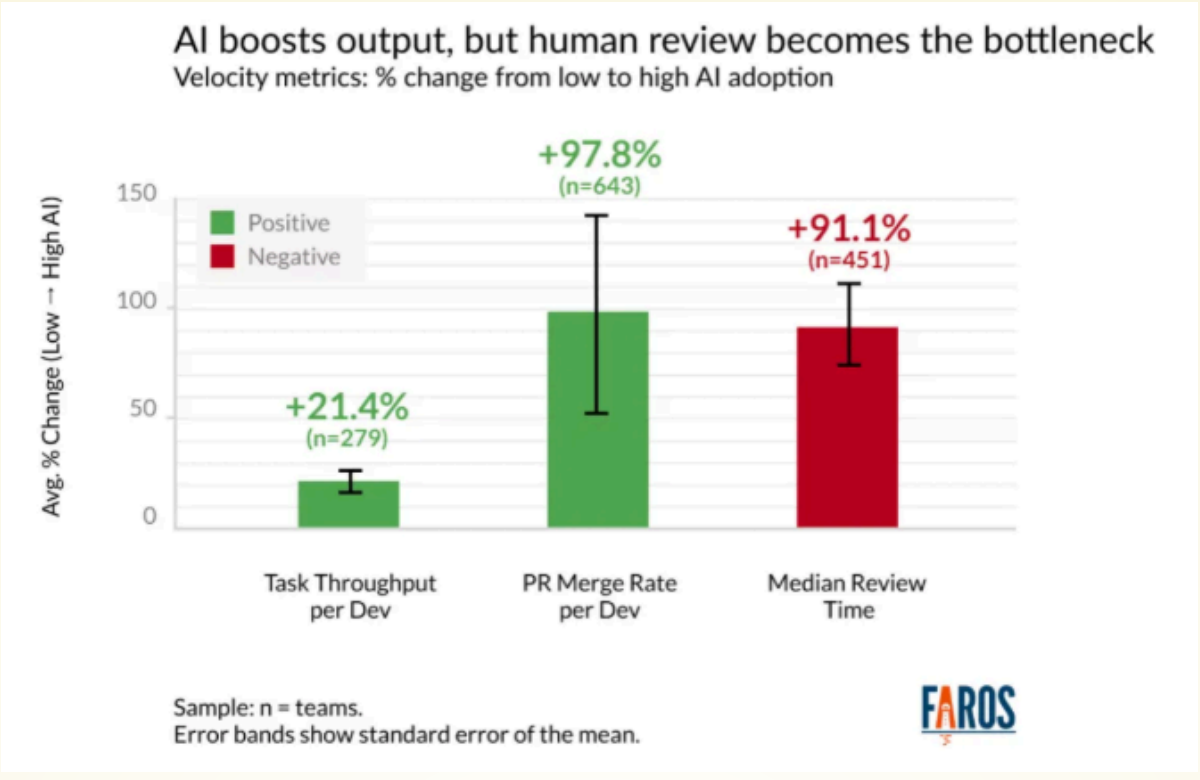
THIS YEAR - OPPORTUNITIES

- Address the stress (reviews, …)
- (Start to) Think different (again) …
  - Build a system to build systems
  - Flying the plane. Building the plane. (Re)Building the factory
- How to scale horizontally?
  - Single-Engineer/Agent vs. Multi-Engineer/Agent

Speaker notes

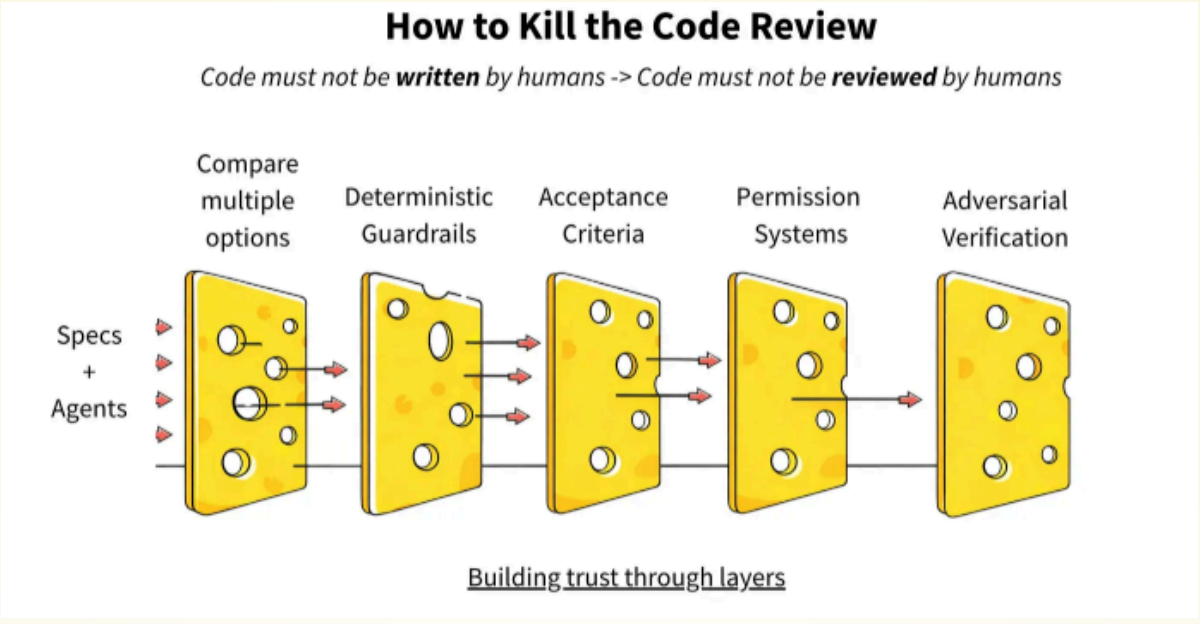
- ???
- ???
- ???

REVIEW THE REVIEW



Source: https://www.latent.space/p/reviews-dead

REVIEW THE REVIEW



Source: https://www.latent.space/p/reviews-dead

THIS YEAR - OPPORTUNITIES

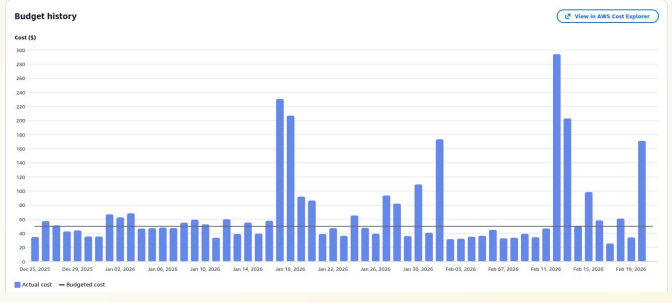
- Address the stress (reviews, …)
- (Start to) Think different (again) …
  - Build a system to build systems
  - Flying the plane. Building the plane. (Re)Building the factory
- How to scale horizontally?
  - Single-Engineer/Agent vs. Multi-Engineer/Agent

Speaker notes

- **Address the stress — start with reviews:** The next slides show the data on this. Reviews are the most acute pain point right now because AI-generated code increases PR volume faster than review capacity grows. The opportunity is to use AI on both sides of that equation: AI generates the code AND AI pre-screens the PR before a human touches it. Static analysis, security patterns, missing test coverage, documentation gaps — all catchable automatically. Your senior engineers' review time then goes to the things only they can judge: architecture, domain correctness, subtle logic. That is a meaningful quality-of-life improvement and a quality improvement at the same time.
- **Three levels of thinking differently:**
  - **Flying the plane** is where most teams are today — using AI tools to write code faster. It is valuable but it is the ground floor.
  - **Building the plane** is the meta-level investment: designing your AI-assisted workflow deliberately. Which tasks go to AI? Which stay human? What does your CLAUDE.md look like? What prompting standards has your team agreed on? What does your review checklist look like for AI-generated code? This is the work most teams skip — and it is where most of the leverage is.
  - **(Re)Building the factory** is the org-level transformation: what does your entire development process look like when agents are first-class participants, not just tools? New roles, new handoffs, new definitions of done. Most orgs are not ready for this yet — but thinking about it now shapes the decisions you make at the other two levels.
- **Horizontal scaling is the unlock for this year:** Today the dominant pattern is one engineer plus one AI assistant — a pair. The emerging pattern is one engineer orchestrating multiple agents in parallel: one exploring the solution space, one writing tests, one reviewing output, one updating docs. That is not a linear productivity gain — it is a different model of work entirely. Start experimenting with this on low-risk, well-defined workstreams now. The engineers who have hands-on experience managing agent parallelism before it becomes the expected norm will have a significant head start.

THIS YEAR - AGENT-MANAGEMENT PLATFORMS

- Kubernetes for Agents
- Not working towards up, but towards done (nothing to do)
- Gastown, Agent-Flywheel, CrewAI, …
- Mixed experience/results (with Gastown)
    - Buggy, patience, cost, …



Source: Gastown

Speaker notes

- **"Kubernetes for Agents" — the analogy is deliberate:** Before Kubernetes, every team ran their own container orchestration and it was a mess. K8s gave us a shared abstraction for managing fleets of services. We are at that same pre-standardisation moment for AI agents. Right now every team is wiring up their own agent loops, their own retry logic, their own cost controls. The platforms in this space — Gastown, CrewAI, Agent-Flywheel — are trying to be that shared abstraction. None of them is K8s yet. But one of them will be.
- **The mental model shift — done, not up:** This is subtle but important. With services, the operational goal is uptime. With agents, the operational goal is completion. You want the agent to finish the task and stop. You are not monitoring heartbeats — you are monitoring outcomes. That requires different tooling, different alerting, different cost models. Your engineers who have spent 10 years thinking in terms of SLAs and uptime need to rewire their intuitions here.
- **Honest about the experience:** The image on the right tells the story. We ran Gastown. The costs were real and visible before the value was. The bugs were frequent. The workflow required patience — a lot of waiting, a lot of retrying, a lot of "why did it do that?" debugging. This is not a reason to avoid it. This is what early adoption looks like. The engineers who tolerate that friction now are building intuition that will be extremely valuable in 12 months when these platforms stabilise.
- **Why this matters for your org this year:** If you are a 100-person engineering organisation, you probably cannot afford to build your own agent orchestration layer. But you can pick one of these platforms, run a pilot on a real workflow, and start accumulating the operational knowledge. The cost of waiting is that when agent platforms mature, you are starting from zero.

BEYOND

- Living in a world with no constrains? An abundance of solutions?
  - You do not need to prioritze anymore. Everything can be built!
- The bottleneck is to describe problems that need solving in a way so that they can be solved (e.g. with a measurable definition-of-done)

Speaker notes

- **The end of scarcity:** Every prioritization framework we have — RICE, MoSCoW, OKRs, the product backlog — exists because implementation capacity is scarce. There are always more ideas than engineers. That assumption is load-bearing for almost every process in a software organisation. What happens when it is no longer true? When an agent fleet can execute in parallel across dozens of workstreams simultaneously? The planning meeting where you argue about what makes the next sprint — that meeting may not exist in five years. That is disorienting and exciting in equal measure.
- **The new bottleneck is human:** If anything can be built, the constraint shifts entirely to knowing what to build — and being able to describe it with enough precision that it can actually be executed. Not a vague user story. Not a Jira ticket with three lines. A description of the problem, the constraints, the edge cases, and a measurable definition of done. The people who can do that well — who can think rigorously about problems before a line of code is written — become the most valuable people in the organisation. This is a profound shift: the profession has spent 50 years being valued for "how." The future values "what" and "why."
- **Definition-of-done is the key unlock:** This is not abstract. A measurable DoD is precisely what makes agent-driven development reliable. If you cannot define "done" in a way that is testable and verifiable, no agent can complete the task correctly — it will optimise for the wrong signal, or stop too early, or never stop. The investment in writing rigorous acceptance criteria today is not just good engineering hygiene. It is the prerequisite for the world described on this slide. Everything connects back to that skill.
- **End on the question, not the answer:** We do not know exactly what this world looks like. Nobody does. But the engineers and organisations that are building the habits now — clear specs, measurable outcomes, deep domain knowledge, comfort with agents — are the ones who will shape it rather than be shaped by it. That is what skating to where the puck is going to be actually means.

WRAP-UP/QUESTIONS

Speaker notes

- **Restate the through-line in one breath:** We started with Wayne Gretzky. The puck is moving fast and in multiple directions at once. Multispeed adoption, counterintuitive productivity data, no established playbook. The engineers and orgs that will win are the ones investing now — in domain knowledge, in precise thinking, in the habits that compound.
- **Three things to take home:**
    - AI does not automatically make experienced engineers more productive. You have to deliberately redesign your workflow — not just add a tool to it.
    - The skills that matter most right now are not technical: they are the ability to read fast, write precisely, and define done rigorously.
    - The window to establish your own adoption framework is open. It will not stay open. Move deliberately, but move.
- **The ASE Meetup:** If this resonated and you want to go deeper — come to the Augmented Software Engineering Meetup on March 26th, 18:00, UCD, L1.03. We go into the details that a 30-minute talk cannot. Practitioners sharing what is actually working, not what vendors are claiming.
- **Then open it up:** Enough from me. What are you seeing in your own teams? Where is the friction? Where are the wins? What question does your leadership keep asking that you do not have a good answer to yet?